

UNIVERSITÀ DEGLI STUDI DI ROMA "TOR VERGATA"

INGEGNERIA DEL SOFTWARE 2

Analisi empirica della qualità del software

REPORT FINALE - APACHE OPENJPA

Dataset creation, defect prediction,
what-if analysis e automated refactoring

Studente: Ali Shalby

Docente: Prof. Davide Falessi

Anno Accademico: 2025/2026

Repository: github.com/Ali-Shalby/openjpa | branch isw2-project

Baseline: Apache OpenJPA 4.1.1 | tag 4.1.1

Indice

1. Introduzione	3
2. Metodologia	4
2.1 Milestone 1 - Dataset Creation	4
2.1.1 Selezione delle release e degli snapshot	4
2.1.2 Inventario delle classi e metriche	4
2.1.3 Code smell, defect e labeling BUGGY	5
2.2 Milestone 2 - Classification	6
2.3 Milestone 3 - What-if Analysis	7
2.4 Milestone 4 - Automated Refactoring	7
3. Risultati	8
3.1 Milestone 1 - Dataset e andamento storico	8
3.2 Milestone 2 - Prestazioni dei classificatori	10
3.3 Milestone 3 - Osservazioni buggy potenzialmente prevenibili	11
3.4 Milestone 4 - Esito del refactoring automatico	13
3.4.1 PCEnhancer	13
3.4.2 ListIteratorWrapper	14
4. Threats to Validity	15
5. Conclusioni	16
Appendice A - Tracciabilità degli artefatti principali	17

1. Introduzione

Il progetto analizza Apache OpenJPA, progetto open-source della Apache Software Foundation, con l'obiettivo di studiare in modo integrato la relazione tra qualità statica del codice, storia dei difetti, predizione della bugginess e refactoring automatico assistito da Large Language Model (LLM). La baseline tecnica è Apache OpenJPA 4.1.1, corrispondente al tag Git 4.1.1. La baseline è stata mantenuta su un branch immutabile e verificata sia tramite build Maven locale sia attraverso la Continuous Integration del progetto; gli artefatti sperimentali, i dataset e la documentazione di supporto sono invece isolati nella directory isw2/ e nel branch isw2-project.

Lo studio è articolato in quattro fasi complementari. La prima ricostruisce un dataset storico a granularità classe-release, integrando metriche di prodotto e di processo, code smell e informazioni di difettosità. La seconda confronta più classificatori e configurazioni di preprocessing per individuare il modello più efficace secondo Precision, Recall, AUC, Kappa e NPofB20. La terza usa il modello selezionato in uno scenario controfattuale in cui NSmells viene azzerata a parità degli altri predictor, misurando la variazione delle prediction BUGGY. La quarta valuta il refactoring automatico su due classi target, variando il contesto di testing fornito all'LLM e verificando congiuntamente compilabilità, test di regressione, code smell e variazioni di metriche snapshot.

L'unità di analisi delle prime tre milestone è l'osservazione classe-release: la stessa classe può quindi comparire in più release con metriche, smell e stato BUGGY differenti. Questa distinzione è importante soprattutto nell'interpretazione della Milestone 3: i valori riportati non rappresentano 423 nomi di classe necessariamente distinti, ma 423 osservazioni classe-release per le quali il conteggio predetto di BUGGY diminuisce nello scenario sintetico. La Milestone 4 restringe invece l'analisi a due classi concrete, PCEnhancer e ListIteratorWrapper, già dotate di artefatti di testing strutturati utilizzabili come contesto sperimentale.

La tracciabilità è trattata come requisito sperimentale: dataset intermedi, output raw, summary, audit e documentazione metodologica sono versionati separatamente. Le scelte di progetto che possono influenzare i risultati - tra cui il criterio di risoluzione release-commit, la tecnica di feature selection, il balancing e l'ordine dei preprocessing - sono esplicitate e motivate. La pipeline evita correzioni manuali dei risultati e conserva gli artefatti necessari a ricostruire le principali trasformazioni dai dati grezzi agli output finali. Il report presenta dapprima metodologia e risultati delle quattro milestone, quindi discute le minacce alla validità e le implicazioni del caso di studio.

La documentazione tecnica e gli artefatti di replicazione sono disponibili nel repository pubblico; l'Appendice A sintetizza i percorsi principali utilizzati per verificare dataset, output sperimentali e analisi M1-M4.

2. Metodologia

2.1 Milestone 1 - Dataset Creation

La Milestone 1 costruisce Dataset A a granularità di classe Java. Ogni riga è identificata dalla coppia (ReleaseIndex, Class) e descrive lo stato della classe in uno specifico snapshot del progetto. La pipeline è stata automatizzata nel modulo Maven indipendente isw2/analyzer, eseguito con Java 21 in modo da non modificare il reactor originale di OpenJPA.

2.1.1 Selezione delle release e degli snapshot

Le release vengono recuperate da Apache JIRA considerando le versioni rilasciate con releaseDate disponibile. Il catalogo raw contiene 42 versioni rilasciate, delle quali 36 rispettano il formato stabile X.Y.Z. Poiché alcuni tag storici risultavano associati a commit successivi alla data ufficiale di release, il tag non è stato utilizzato come snapshot definitivo. Per ogni release stabile è stato invece adottato il criterio DATE_CUTOFF: viene scelto l'ultimo commit raggiungibile dalla baseline la cui data non sia successiva alla ReleaseDate. La proprietà ReleaseCommitDate <= ReleaseDate è stata verificata per tutte le 36 release stabili.

Dataset A utilizza la prima porzione pari al 33% delle release stabili. Applicando $\text{ceil}(36 \times 0,33)$ si ottengono 12 release, comprese tra OpenJPA 0.9.0 (26 agosto 2006) e 2.0.0 (22 aprile 2010). La selezione opera sulla cronologia, non sull'ordinamento lessicografico delle versioni.

2.1.2 Inventario delle classi e metriche

Per ogni commit di release l'albero Git viene letto direttamente con git ls-tree, senza checkout ripetuti. I file Java sono classificati esplicitamente in PRODUCTION, TEST, EXAMPLE, GENERATED, PARSER_SOURCE, NON_CLASS e OTHER. Questa classificazione evita di assumere che ogni file sotto src/main/java sia automaticamente production: due sorgenti del modulo OSGi integration testing, ad esempio, sono stati riclassificati come TEST. L'inventario finale contiene esclusivamente le classi production.

Per ciascuna osservazione vengono calcolate metriche snapshot e storiche. LOC misura le linee fisiche di codice, escludendo linee vuote e commenti. Le metriche storiche sono ricostruite con git log --follow --no-merges --numstat fino allo snapshot della release e comprendono, tra le altre, numero di revisioni (NR), autori distinti (NAUTH), LOC touched e added, churn, change-set size, age e weighted age. Una revisione incrementa NR solo quando added + deleted > 0: rename o cambi di file mode senza modifica testuale vengono seguiti ma non contati come revisione sorgente effettiva. NFIX viene calcolata successivamente a partire dai defect ticket e dai relativi fix commit.

Le 18 feature predittive di Dataset A sono riassunte in Tabella 1. Le prime sedici descrivono dimensione e storia evolutiva della classe; NSmells quantifica il debito statico osservato nello snapshot e NFIX sintetizza la storia dei defect fixing. Le tre colonne identificative (Project, Class, ReleaseIndex) non sono predictor, mentre BUGGY costituisce la variabile target.

Metrica	Definizione operativa	Interpretazione
LOC	Linee fisiche di codice Java nello snapshot, escluse linee vuote e commenti.	Dimensione corrente della classe.
LOC_TOUCHED	Somma cumulativa di added + deleted nelle revisioni effettive fino alla release.	Volume complessivo di codice modificato.
NR	Numero di revisioni effettive con added + deleted > 0.	Frequenza di modifica del sorgente.
NAUTH	Numero di autori distinti che hanno modificato effettivamente la classe.	Ampiezza della ownership storica.
LOC_ADDED	Somma cumulativa delle LOC aggiunte nelle revisioni effettive.	Quantità storica di codice introdotto.
MAX_LOC_ADDED	Massimo numero di LOC aggiunte osservato in una singola revisione effettiva.	Picco di crescita in una modifica.

Metrica	Definizione operativa	Interpretazione
AVG_LOC_ADDED	LOC_ADDED / NR.	Aggiunte medie per revisione.
CHURN	Somma cumulativa di added - deleted.	Variazione netta del codice nel tempo.
MAX_CHURN	Massimo churn osservato in una singola revisione effettiva.	Picco di variazione netta.
AVG_CHURN	CHURN / NR.	Variazione netta media per revisione.
CHANGE_SET_SIZE	Somma del numero di file modificati nei commit associati alle revisioni effettive.	Ampiezza cumulativa dei change set.
MAX_CHANGE_SET	Massimo numero di file modificati in uno dei commit della storia della classe.	Massima ampiezza di una modifica correlata.
AVG_CHANGE_SET	CHANGE_SET_SIZE / NR.	Ampiezza media dei change set.
AGE_WEEKS	Settimane tra la release osservata e la prima revisione effettiva della classe.	Età della classe nello snapshot.
WEIGHTED_AGE_WEEKS	Media dell'età delle revisioni pesata per LOC_TOUCHED.	Età pesata verso le modifiche più estese.
IGNORED_ZERO_LOC_REVS	Numero di revisioni Git incontrate ma escluse da NR perché non modificano LOC del sorgente.	Indicatore diagnostico di rename/mode-only o revisioni senza variazione testuale.
NSmells	Numero di issue SonarQube OPEN di tipo CODE_SMELL associate alla classe nella release.	Debito statico osservato nello snapshot.
NFIX	Numero cumulativo di IssueKey JIRA distinti associati a fix commit nella storia della classe.	Storia dei defect fixing che hanno interessato la classe.

Tabella 1 - Definizione operativa delle 18 feature predittive di Dataset A.

2.1.3 Code smell, defect e labeling BUGGY

NSmells viene calcolata tramite SonarQube Cloud contando, per ogni classe e release, le issue OPEN di tipo CODE_SMELL appartenenti alla qualità MAINTAINABILITY. La raccolta conserva sia il conteggio aggregato sia le evidenze delle issue, e applica una join 1:1 con le metriche di classe sulla chiave (ReleaseIndex, Class).

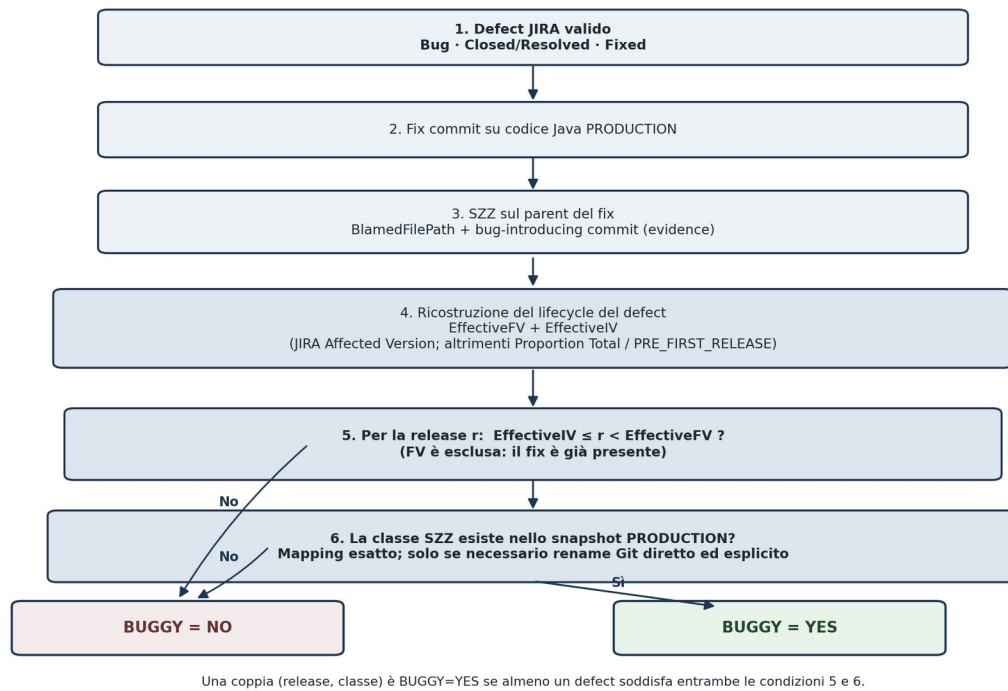
La ricostruzione dei defect parte dai ticket JIRA e dai fix commit. SZZ viene applicato ai fix che modificano codice Java production per individuare le linee blameabili e i bug-introducing commit. Su 1.222 mapping fix processati, 943 producono evidenze SZZ; l'insieme delle evidenze riguarda 766 defect e 982 bug-introducing commit. I casi senza linee blameabili vengono conservati come NO_EVIDENCE invece di essere confusi con errori di pipeline.

Per determinare l'intervallo temporale del defect si distinguono Injected Version (IV), Opening Version (OV) e Fixed Version (FV). Quando JIRA fornisce una Affected Version coerente, questa è utilizzata come IV osservata. Negli altri casi viene applicato Proportion Total sulla timeline completa delle 42 release JIRA rilasciate, evitando di comprimere le distanze temporali alle sole release del Dataset A. Per i defect utilizzabili vale $P = (FV - IV) / (FV - OV)$, con gestione specifica del caso $OV = FV$; il valore aggregato ottenuto è $P_TOTAL = 1,9688220484$. L'IV stimata è poi $EffectiveIV = \text{ceil}(EffectiveFV - \text{distance} \times P_TOTAL)$, con clamp alla prima release quando necessario.

La FV effettiva usa JIRA come fonte primaria, ma viene verificata rispetto alla raggiungibilità dei fix commit negli snapshot Git. In assenza di una Fix Version utilizzabile, o quando la versione JIRA non contiene i fix osservati, si usa come fallback la prima release RAW che contiene il fix commit. Tra i 766 defect con SZZ, 696 ottengono FV da JIRA, 36 da Git per assenza di FV JIRA utilizzabile e 34 da Git per mismatch JIRA/SZZ; non restano FV irrisolte. L'intervallo di vita è $[EffectiveIV, EffectiveFV]$: la release che contiene il fix è esclusa dalla bugginess.

Un'osservazione viene marcata BUGGY=YES quando esiste almeno un defect attivo nella release e la classe associata all'evidenza SZZ appartiene all'inventario production dello snapshot. Il mapping usa in prima istanza il path esatto; un path storico alternativo viene accettato solo in presenza di un rename Git esplicito, evitando matching fuzzy sul solo nome del file. Dataset A viene infine

assemblato tramite join validate fra metriche, NSmells, NFIX e label BUGGY. La logica complessiva del labeling è sintetizzata nello Schema 1.



Schema 1 - Processo di determinazione della buginess per una coppia (release, classe).

Nota metodologica: il bug-introducing commit prodotto da SZZ è conservato come evidenza della classe coinvolta; la finestra temporale del labeling è determinata dal lifecycle [EffectiveIV, EffectiveFV], con EffectiveFV esclusa.

Artefatti: isw2/datasets/release_catalog.csv, java_class_inventory*.csv, class_metrics*.csv, sonar_smell_metrics.csv, nfix_metrics.csv, szz_evidence.csv, buginess_labels.csv, openjpa_dataset_a.csv; metodologia completa in isw2/docs/milestone1.md.*

2.2 Milestone 2 - Classification

La Milestone 2 usa l'intero Dataset A finale: 12.836 osservazioni, 18 predictor numerici e BUGGY come classe binaria. Sono confrontati tre classificatori eterogenei - RandomForest, NaiveBayes e IBk - implementati con Weka 3.8.7. Non viene eseguito hyperparameter tuning: vengono utilizzati i parametri di default Weka, con seed espliciti per le componenti randomiche in modo da garantire la riproducibilità.

La validazione principale è una 10 times 10-fold cross-validation. Ogni repetition usa un seed indipendente da 1 a 10; il piano dei dieci fold viene generato una sola volta per repetition e riutilizzato identico per tutti i classificatori e tutte le configurazioni di preprocessing. La cross-validation è stratificata rispetto a BUGGY. Le prediction dei dieci test fold vengono aggregate in un unico insieme out-of-fold e le metriche della repetition sono calcolate sull'insieme aggregato, non come semplice media delle metriche dei singoli fold.

Sono state progettate quattro configurazioni per isolare l'effetto dei due preprocessing: C1 senza feature selection e senza balancing; C2 con feature selection e senza balancing; C3 senza feature selection e con balancing; C4 con entrambi. La feature selection utilizza CfsSubsetEval + BestFirst, mentre il balancing usa SMOTE. Quando entrambi sono attivi, l'ordine è Feature Selection -> SMOTE -> Classifier, in modo che il subset di feature sia appreso sulle sole osservazioni reali del training e non sulle istanze sintetiche.

Config. M2	Feature Selection	Balancing	Scopo
C1	No	No	baseline senza preprocessing

Config. M2	Feature Selection	Balancing	Scopo
C2	Sì	No	effetto della sola feature selection
C3	No	SMOTE	effetto del solo balancing
C4	Sì	SMOTE	effetto combinato

Tabella 2 - Disegno sperimentale della Milestone 2.

Tutti i preprocessing che apprendono dai dati vengono stimati esclusivamente sul training fold. Il test fold non viene mai sottoposto a oversampling e non influenza la selezione delle feature. Prima dell'esecuzione FULL è stata eseguita una modalità QUICK end-to-end per verificare loader, fold planner, filtri, classificatori, prediction, metriche e controlli anti-leakage; i risultati QUICK non contribuiscono alle conclusioni sperimentali.

L'esecuzione FULL comprende 10 repetition x 10 fold x 4 configurazioni x 3 classificatori, pari a 1.200 training di modelli e 120 righe raw di metriche. Le cinque metriche considerate sono Precision, Recall, AUC, Cohen's Kappa e NPofB20. Quest'ultima ordina le osservazioni per P(BUGGY=YES)/LOC e misura la quota di osservazioni realmente buggy individuata entro un budget di ispezione pari al 20% delle LOC complessive.

Artefatti: `isw2/results/m2/*` e `isw2/docs/milestone2.md`; Weka 3.8.7, configurazioni e controlli anti-leakage documentati nel runner.

2.3 Milestone 3 - What-if Analysis

La Milestone 3 usa il classificatore vincente di M2 per rispondere alla domanda: quante osservazioni buggy potrebbero cambiare previsione se gli smell fossero pari a zero? Dal Dataset A vengono costruiti tre insiemi: B+ contiene le osservazioni reali con NSmells > 0; C contiene le osservazioni reali con NSmells = 0; B è una copia sintetica di B+ nella quale l'unica modifica è NSmells = 0. Identificatori, tutti gli altri predictor e la label storica BUGGY rimangono invariati tra B+ e B.

B+ = A con NSmells > 0
 C = A con NSmells = 0
 B = copia di B+ con il solo NSmells impostato a 0

Il modello finale BClassifierA è RandomForest, selezionato nella Milestone 2. Viene addestrato sull'intero Dataset A con tutti i 18 predictor, senza Feature Selection e senza SMOTE, seed 1. NSmells viene mantenuto esplicitamente tra i predictor perché costituisce la variabile manipolata nel passaggio B+ -> B. La valutazione di accuratezza rimane quella out-of-sample della M2; le prediction M3 su A sono in-sample e vengono usate solo come controllo descrittivo e per la simulazione controfattuale.

Il risultato viene misurato in due modi complementari: variazione del numero complessivo di predicted BUGGY=YES e transizioni riga-per-riga B+ -> B. Poiché RandomForest non impone monotonicità rispetto a una singola feature, sono possibili sia transizioni YES->NO sia NO->YES; la stima finale usa quindi la riduzione netta, non solo il conteggio delle transizioni favorevoli.

Artefatti: dataset B+, B, C e prediction M3 sotto `isw2/datasets` e `isw2/results`; metodologia in `isw2/docs/milestone3.md`.

2.4 Milestone 4 - Automated Refactoring

La Milestone 4 verifica se sia possibile ottenere un refactoring automatico di qualità migliore variando l'informazione di testing resa disponibile all'LLM. Le classi analizzate sono `org.apache.openjpa.enhance.PCEnhancer` e `org.apache.openjpa.lib.util.collections.ListIteratorWrapper`. Per ogni classe viene fissata una baseline C0 e vengono generate quattro varianti indipendenti.

Variante M4	Informazione di testing fornita al modello
C1	nessun test
C2	T_BB - test black-box da Category Partition
C3	T_BB + T_CF - test aggiunti per coprire gap di control flow
C4	T_BB + T_CF + T_MT - ulteriori test derivati da mutation analysis

Tabella 3 - Condizioni sperimentali della Milestone 4.

Le suite automatiche Randoop, EvoSuite e LLM-test non vengono fornite come vincolo di generazione delle varianti. Ogni C1-C4 nasce nuovamente da C0 in una conversazione indipendente, senza usare le varianti precedenti o i relativi errori. Dopo la generazione il sorgente viene congelato e valutato senza cicli di repair iterativo: in questo modo la compilabilità stessa diventa un outcome sperimentale e non un obiettivo corretto post-hoc.

Il prompt fornisce il sorgente della classe e le informazioni Sonar, richiede un refactoring conservativo orientato alla manutenibilità e impone di preservare comportamento osservabile, API pubblica, package e compatibilità col progetto. La quantità di informazione di test cambia secondo C1-C4. Per ListIteratorWrapper, che in C0 presenta già NSmells=0, l'obiettivo non viene forzato artificialmente come "riduzione degli smell", ma formulato come miglioramento ragionevole della manutenibilità senza introdurre regressioni.

La valutazione segue quattro domande operative: (1) la variante compila nel reale modulo Maven OpenJPA? (2) quali code smell rimangono, vengono risolti o vengono introdotti rispetto a C0? (3) aumenta qualche feature snapshot positivamente correlata a BUGGY? (4) aumenta qualche feature snapshot negativamente correlata? Per il confronto delle issue Sonar, C0 e C1-C4 vengono analizzate con regole correnti omogenee e IssueKey viene usata come identità operativa old/resolved/new.

Le correlazioni con BUGGY vengono calcolate sull'intero Dataset A usando Pearson con BUGGY codificata 0/1, equivalente alla correlazione point-biserial nel caso binario. Tra le feature direttamente modificabili da un refactoring locale, LOC e NSmells risultano entrambe positivamente correlate con BUGGY, rispettivamente circa +0,3748 e +0,3527. Le metriche storiche e di processo restano invece invariate, poiché C1-C4 sono sostituzioni controfattuali del sorgente e non nuove release storiche.

Artefatti: [isw2/docs/m4/pcenhancer-m4-analysis.md](#) e [listiteratorwrapper-m4-analysis.md](#), con sorgenti frozen, SHA-256, compile evidence e Sonar evidence sotto [isw2/results](#).

3. Risultati

3.1 Milestone 1 - Dataset e andamento storico

La pipeline produce 12.836 osservazioni production distribuite sulle 12 release selezionate. Il numero di classi passa da 932 in R1 a 1.293 in R12, con una crescita complessiva di circa il 38,7%, pur con piccole oscillazioni locali. La dimensione del sistema appare quindi sostanzialmente crescente nel periodo osservato. L'inventario finale presenta zero chiavi duplicate, zero valori mancanti e nessuna osservazione appartenente agli scope esclusi. La distribuzione per release è riportata in Tabella 4 e visualizzata, insieme al tasso di bugginess, in Figura 1.

Rel.	Versione	Data	Classi prod.	BUGGY=YES	Buggy rate
R1	0.9.0	26/08/2006	932	63	6,76%
R2	0.9.6	29/11/2006	949	100	10,54%

Rel.	Versione	Data	Classi prod.	BUGGY=YES	Buggy rate
R3	0.9.7	27/04/2007	948	158	16,67%
R4	1.0.0	28/08/2007	996	123	12,35%
R5	1.0.1	09/11/2007	1.029	113	10,98%
R6	1.0.2	18/02/2008	1.058	136	12,85%
R7	1.1.0	22/05/2008	1.045	232	22,20%
R8	1.0.3	23/07/2008	1.050	234	22,29%
R9	1.2.0	12/08/2008	1.051	210	19,98%
R10	1.2.1	18/03/2009	1.185	350	29,54%
R11	1.2.2	18/01/2010	1.300	159	12,23%
R12	2.0.0	22/04/2010	1.293	132	10,21%

Tabella 4 - Distribuzione delle osservazioni e della bugginess per release.

Evoluzione della dimensione del sistema e della bugginess

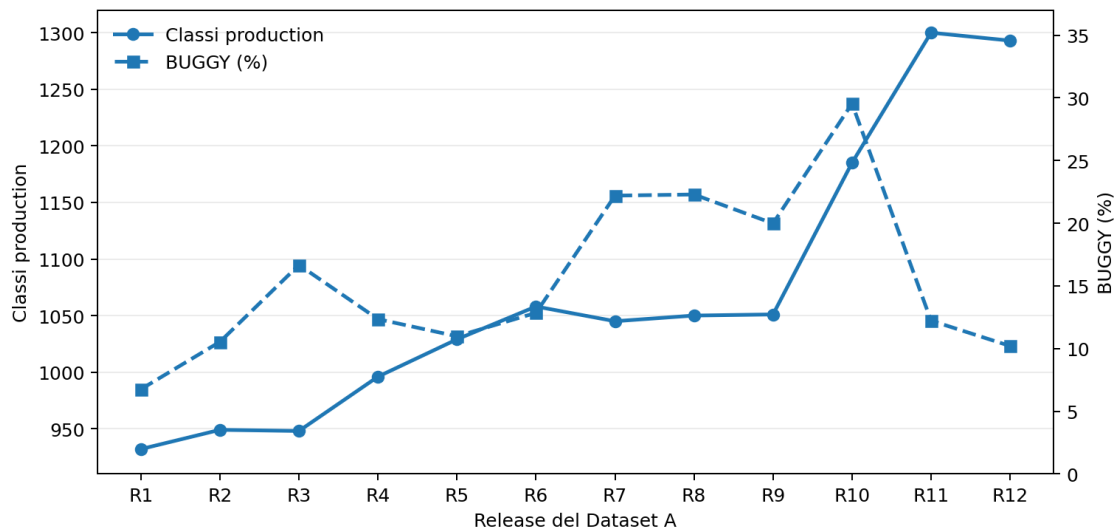


Figura 1 - Evoluzione del numero di classi production e del tasso di osservazioni BUGGY nelle 12 release.

Sul totale, 2.010 osservazioni sono etichettate BUGGY=YES e 10.826 BUGGY=NO, per un buggy rate del 15,66%. La distribuzione non è stabile nel tempo: il tasso cresce dal 6,76% di R1 fino a valori superiori al 22% in R7-R8, raggiunge il massimo del 29,54% in R10 e poi scende al 12,23% in R11 e al 10,21% in R12. Il forte calo R10->R11 è stato sottoposto ad audit specifico: 250 path cessano di essere buggy e tutti sono supportati esclusivamente da defect che escono dal lifecycle prima di R11, senza perdite arbitrarie di mapping.

Il picco di R10 è inoltre spiegato in larga misura da OPENJPA-896, defect trasversale relativo agli end-of-line: esso contribuisce da solo a 246 delle 350 osservazioni BUGGY di R10. L'audit ha confermato che il fenomeno deriva da un fix cross-cutting che tocca molti file, non da un errore di conteggio. Il ticket è stato mantenuto nel dataset poiché la pipeline non applica filtri soggettivi basati sulla natura funzionale o di formattazione del defect.

Sul fronte dei code smell, l'analisi Sonar delle 12 release produce 94.308 evidenze di maintainability code smell; la somma di NSmells coincide esattamente con tale conteggio. L'assemblaggio finale contiene 18 feature predittive - metriche di classe e di processo, NSmells e NFIX - più la label

BUGGY. La somma di NFIX è 7.523. La validazione finale conferma 12.836 chiavi uniche, 12 release, nessun mismatch tra dataset intermedi e ValidationPassed=True.

Indicatore Dataset A	Valore
Osservazioni classe-release	12.836
Release	12
Predictor	18
Sum(NSmells)	94.308
Sum(NFIX)	7.523
BUGGY=YES	2.010 (15,66%)
BUGGY=NO	10.826 (84,34%)
Validazione finale	PASS

Tabella 5 - Sintesi del Dataset A finale.

3.2 Milestone 2 - Prestazioni dei classificatori

RandomForest risulta il miglior classificatore per tutte e cinque le metriche considerate. Non cambia quindi l'algoritmo vincente al variare della metrica; cambia invece la configurazione di preprocessing ottimale. La configurazione C1, senza feature selection e senza balancing, massimizza Precision e NPofB20; C3, con SMOTE ma senza feature selection, massimizza Recall, AUC e Kappa. I migliori valori medi sono riepilogati in Tabella 6 e Figura 2.

Metrica	Configurazione migliore	Classificatore	Valore medio
Precision	C1 - no FS, no balancing	RandomForest	0,842343
Recall	C3 - no FS, SMOTE	RandomForest	0,793881
AUC	C3 - no FS, SMOTE	RandomForest	0,963241
Kappa	C3 - no FS, SMOTE	RandomForest	0,750842
NPofB20	C1 - no FS, no balancing	RandomForest	0,691393

Tabella 6 - Migliori risultati globali della Milestone 2.

Miglior risultato globale per metrica - RandomForest

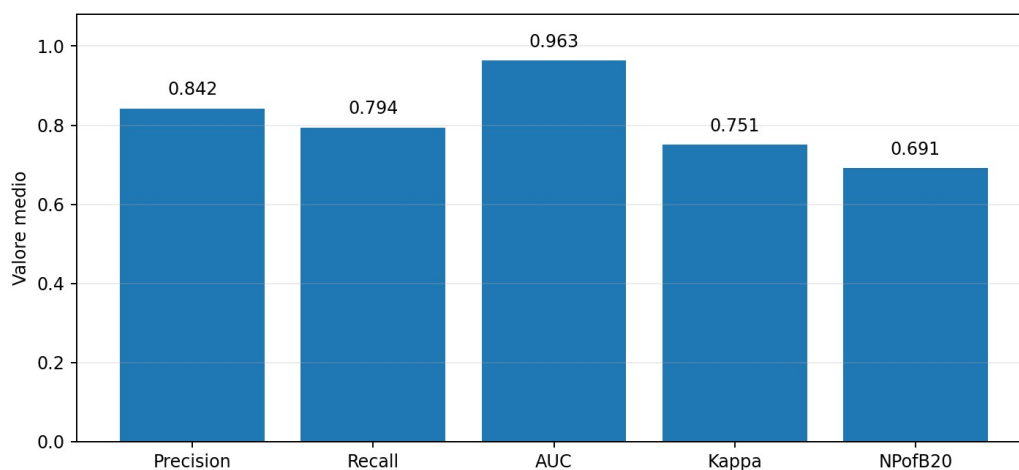


Figura 2 - Valore medio della migliore configurazione per ciascuna metrica.

L'effetto di SMOTE è coerente con il trade-off tipico precision/recall. Passando da C1 a C3 su RandomForest, Recall aumenta di 0,072239, AUC di 0,002592 e Kappa di 0,011123, mentre Precision diminuisce di 0,056090. NPofB20 rimane praticamente invariata: 0,691393 in C1 contro 0,690995 in C3, una differenza media di appena 0,000398. In termini operativi, il balancing permette quindi di recuperare più osservazioni buggy, accettando un maggior numero di falsi positivi, senza migliorare in modo apprezzabile l'efficienza del ranking entro il 20% delle LOC.

Il risultato più sorprendente riguarda la feature selection. CfsSubsetEval + BestFirst riduce tutte le metriche di RandomForest rispetto alla configurazione equivalente che mantiene i 18 predictor, sia senza balancing sia con SMOTE. Senza balancing (C2-C1), la perdita è -0,104176 in Precision, -0,051542 in Recall, -0,018343 in AUC, -0,089341 in Kappa e -0,017264 in NPofB20. Con balancing (C4-C3) il segno rimane negativo per tutte le metriche. Il dato non invalida la feature selection come tecnica e, da solo, non identifica la causa del peggioramento: mostra però che, su questo Dataset A e con questa configurazione di RandomForest, il sottoinsieme selezionato non migliora le prestazioni ed è compatibile con la perdita di parte del segnale predittivo utile.

Per le milestone successive il BClassifier viene quindi identificato con RandomForest. La scelta riguarda il classificatore, non una singola configurazione di preprocessing universale: i risultati mostrano esplicitamente che l'assetto migliore dipende dall'obiettivo metrico.

3.3 Milestone 3 - Osservazioni buggy potenzialmente prevenibili

La partizione di A produce 8.933 osservazioni in B+ (NSmells>0), pari al 69,59% del dataset, e 3.903 osservazioni già prive di smell in C, pari al 30,41%. In B+ ricadono 1.723 delle 2.010 osservazioni storicamente BUGGY, cioè l'85,72% del totale. B mantiene esattamente le stesse 8.933 righe di B+ e differisce soltanto per NSmells, impostata a zero. La composizione dei dataset e le prediction aggregate sono riportate in Tabella 7.

Dataset	Righe	Reference YES	Predicted YES	Mean P(YES)
A	12836	2010	2010	0,154853
B+	8933	1723	1723	0,189733
B	8933	1723*	1300	0,162006
C	3903	287	287	0,075023

Tabella 7 - Prediction del modello finale sui dataset della Milestone 3. *Per B la reference è la label storica della corrispondente riga B+.

Effetto controfattuale della rimozione degli smell

Predicted BUGGY=YES: 1.723 -> 1.300 (riduzione netta = 423)

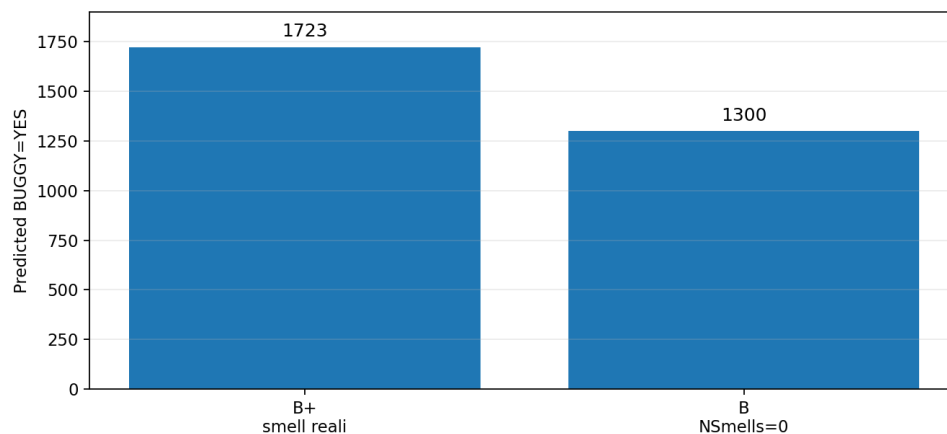


Figura 3 - Confronto tra B+ reale e B sintetico dopo l'azzeramento di NSmells.

Nel passaggio B+ -> B, le predicted BUGGY=YES scendono da 1.723 a 1.300, come mostrato in Figura 3. Il confronto riga-per-riga mostra 425 transizioni YES->NO e 2 transizioni NO->YES; la variazione netta è quindi 423. La probabilità media di BUGGY diminuisce da 0,189733 a 0,162006, con una riduzione media pari a circa 0,027727.

La stima finale è dunque di 423 osservazioni classe-release potenzialmente prevenibili nello scenario zero-smell. Esse rappresentano il 21,04% di tutte le 2.010 osservazioni BUGGY di A e il 24,55% delle 1.723 osservazioni BUGGY appartenenti alla porzione B+ effettivamente sottoponibile all'intervento. Il termine "prevenibile" deve essere letto in senso strettamente controfattuale rispetto al modello: non dimostra che la rimozione reale dei code smell avrebbe causato la scomparsa del defect.

Indicatore what-if	Valore
B+ predicted YES -> B predicted NO	425
B+ predicted NO -> B predicted YES	2
Riduzione netta predicted BUGGY	423
Quota su tutte le BUGGY di A	21,04%
Quota sulle BUGGY di B+	24,55%
Delta mean P(BUGGY)	-0,027727

Tabella 8 - Sintesi quantitativa della what-if analysis.

3.4 Milestone 4 - Esito del refactoring automatico

3.4.1 PCEnhancer

PCEnhancer costituisce il caso più difficile. La baseline C0 compila, contiene 3.958 LOC e, nell'analisi Sonar uniforme della M4, 190 code smell distribuiti su 28 regole. Tutte e quattro le varianti generate dall'LLM riducono NSmells, ma nessuna compila nel reale modulo openjpa-kernel. C1 e C2 introducono chiamate a `InsnList.isEmpty()` non disponibili nella versione ASM usata dal progetto; C3 introduce una incompatibilità `String -> String[]` nella costruzione di un `MethodNode`; C4 lascia non inizializzato il campo `final repos`.

Var.	Compile	Test post-hoc	LOC	NSmells	Delta smell vs C0
C0	PASS	baseline	3958	190	0
C1	FAIL	NOT RUN	3986	138	-52 (-27,37%)
C2	FAIL	NOT RUN	3986	141	-49 (-25,79%)
C3	FAIL	NOT RUN	3961	162	-28 (-14,74%)
C4	FAIL	NOT RUN	3961	167	-23 (-12,11%)

Tabella 9 - Risultati M4 per PCEnhancer.

Dal punto di vista statico, C1 ottiene la riduzione maggiore: 138 smell, cioè -52 rispetto a C0 (-27,37%). C2 scende a 141 (-25,79%), C3 a 162 (-14,74%) e C4 a 167 (-12,11%). Tuttavia le varianti aumentano tutte LOC, una feature positivamente correlata a BUGGY: +28 LOC in C1-C2 e +3 in C3-C4. L'esito complessivo è quindi negativo: una riduzione apparente del debito statico non è sufficiente se il refactoring non conserva neppure la compilabilità. Per questa ragione i test dinamici sono correttamente classificati NOT RUN / BLOCKED BY COMPILATION, non FAIL.

3.4.2 ListIteratorWrapper

ListIteratorWrapper offre un caso complementare. C0 è piccola (134 LOC) e parte già da NSmells=0; tutte le varianti C1-C4 compilano e tutte superano 19/19 test manuali post-hoc. L'LLM non aveva quindi margine per ridurre numericamente gli smell e il risultato va letto come capacità di preservare o peggiorare una baseline già pulita.

Var.	Compile	Test post-hoc	LOC	NSmells	Delta smell vs C0
C0	PASS	baseline	134	0	0
C1	PASS	19/19 PASS	134	1	+1
C2	PASS	19/19 PASS	134	0	0
C3	PASS	19/19 PASS	132	1	+1
C4	PASS	19/19 PASS	139	0	0

Tabella 10 - Risultati M4 per ListIteratorWrapper.

C1 e C3 introducono una singola issue Sonar java:S1125 (boolean literal non necessario), mentre C2 e C4 rimangono a zero smell. C1 mantiene 134 LOC ma aumenta NSmells; C3 riduce LOC a 132 ma aumenta NSmells; C4 mantiene zero smell ma sale a 139 LOC. C2 mantiene sia LOC=134 sia NSmells=0, compila e supera 19/19 test: è quindi la variante con il profilo migliore rispetto ai criteri osservati in questa singola esecuzione.

Il confronto tra le condizioni non mostra un miglioramento monotono all'aumentare del contesto di testing. Sul caso PCEnhancer tutte le condizioni falliscono la compilazione nonostante riduzioni statiche differenti; su ListIteratorWrapper C2, che riceve soltanto T_BB, è più favorevole di C3 e C4 rispetto alla combinazione LOC/NSmells, ma l'esperimento utilizza una sola generazione per condizione e non consente di attribuire causalmente il risultato alla quantità di test fornita. Il dato più robusto è piuttosto metodologico: compilazione, test post-hoc e analisi statica devono essere valutati congiuntamente; nessuna delle tre dimensioni sostituisce le altre.

4. Threats to Validity

Le minacce alla validità sono discusse distinguendo il limite, la possibile conseguenza sui risultati e la mitigazione adottata. Dove il limite deriva direttamente da una scelta sperimentale documentata, viene esplicitato il relativo trade-off; dove non può essere eliminato, viene indicato come direzione per una replica futura.

Tipo	Minaccia / conseguenza	Mitigazione e interpretazione
Construct	Label BUGGY dipendente da JIRA, SZZ, Proportion e mapping dei path. Errori in IV/FV o nel blame possono produrre falsi positivi/negativi.	Evidence SZZ completa, audit dei fix, fallback Git containment per FV, nessuna FV irrisolta, mapping esatto e rename Git esplicito; nessun fuzzy matching.
Construct	NSmells dipende dalle regole e dalla configurazione di Sonar; il numero di smell non coincide con una misura assoluta di qualità.	Definizione operativa unica (OPEN, MAINTAINABILITY, CODE_SMELL), evidence conservata; in M4 C0 e varianti sono rianalizzate con regole omogenee.
Internal	La 10x10 CV di M2 è stratificata ma non impone separazione temporale fra release; versioni diverse della stessa classe possono comparire in train e test.	Fold identici fra configurazioni e preprocessing train-only eliminano il leakage dei filtri, ma non quello potenziale dovuto alla dipendenza temporale. Una replica dovrebbe aggiungere validazione time-aware/release-wise.
Internal	M3 addestra BClassifierA su tutto A e applica il modello anche ad A; tali prediction sono in-sample.	Il progetto non le usa come nuova stima di accuratezza: la generalizzazione è valutata esclusivamente con le prediction out-of-fold di M2.
Causal	B modifica solo NSmells mantenendo fissi tutti gli altri predictor. Il risultato misura la risposta del modello, non l'effetto causale di un refactoring reale.	Interpretazione esplicitamente controfattuale; si usa la riduzione netta 425-2=423 e non si assume monotonicità o causalità.
Conclusion	In M2 non viene eseguito hyperparameter tuning; i risultati confrontano implementazioni Weka con parametri di default.	Scelta uniforme e riproducibile che riduce gradi di libertà, ma può sottostimare il potenziale di singoli classificatori. Una replica può introdurre tuning nested.
Conclusion	La M2 usa l'intero Dataset A e non costruisce dataset incrementali per numero di release.	Scelta coerente con il confronto sul Dataset A completo; non viene quindi studiata la stabilità del ranking dei classificatori al crescere della storia disponibile.
M4	Una sola generazione LLM per C1-C4 e per classe non consente inferenza statistica sull'effetto del contesto di test.	Conversazioni indipendenti, stessa C0, freeze immediato e nessun repair riducono contaminazioni tra condizioni; future repliche dovrebbero usare più run/seed di generazione.
M4	Le quattro varianti PCEnhancer non compilano, quindi non è possibile valutarne comportamento dinamico post-hoc.	La compilazione viene trattata come condizione e outcome; i test sono classificati BLOCKED, evitando di produrre pseudo-risultati su bytecode inesistente.
M4	La suite post-hoc di ListIteratorWrapper (19 test) aumenta la confidenza ma non prova equivalenza semantica completa.	Le varianti vengono validate nel progetto reale, con suite manuale congelata e Sonar uniforme; la conclusione resta limitata al comportamento osservato.
M4 provenance	Per PCEnhancer C1 la risposta completa della conversazione Copilot non è stata preservata.	La variante non è stata rigenerata né ricostruita artificialmente; restano sorgente C1 congelato, input disponibili e compile evidence.
External	Un solo progetto (OpenJPA), un Dataset A riferito alla parte iniziale della storia e due sole classi in M4 limitano la generalizzabilità.	I risultati sono formulati come evidenza sul caso di studio. Servono repliche su altri progetti ASF, periodi storici e classi con profili diversi.

Tabella 11 - Principali minacce alla validità e strategie di mitigazione.

Due trade-off meritano particolare attenzione. Il primo è DATE_CUTOFF nella M1: usare il tag sarebbe stato più semplice, ma avrebbe introdotto snapshot temporalmente successivi alla release in alcuni casi; la scelta del commit non successivo alla ReleaseDate privilegia coerenza temporale rispetto alla semantica nominale del tag. Il secondo è l'assenza di tuning nella M2: limita l'ottimizzazione, ma rende il confronto più controllato e riduce il rischio di adattare manualmente i modelli al dataset.

Anche M4 contiene un trade-off deliberato: le varianti non vengono corrette dopo la prima generazione. Un workflow industriale userebbe certamente compilazione e test come feedback per iterare; qui, invece, il freeze serve a osservare la qualità del primo output nelle quattro condizioni sperimentali. La conseguenza è una percentuale di successo più bassa su classi complesse, ma il risultato conserva il significato dell'esperimento invece di misurare l'abilità di un processo di repair interattivo.

5. Conclusioni

Le quattro milestone costruiscono una catena coerente che parte dalla storia reale di Apache OpenJPA e arriva alla valutazione di trasformazioni automatiche del sorgente. La Milestone 1 ha prodotto un Dataset A validato con 12.836 osservazioni classe-release, 18 predictor e 2.010 osservazioni BUGGY. La bugginess non è uniforme nel tempo: il tasso medio è 15,66%, ma il picco del 29,54% in R10 evidenzia come defect trasversali possano alterare significativamente la distribuzione e richiedano audit qualitativo oltre al semplice conteggio.

La Milestone 2 mostra che RandomForest è il classificatore più forte nel caso di studio per tutte le metriche considerate. L'assenza di un singolo preprocessing universalmente migliore è però altrettanto importante: SMOTE aumenta sensibilmente Recall e migliora leggermente AUC/Kappa, riducendo Precision, mentre la feature selection scelta peggiora tutte le metriche di RandomForest. Per un practitioner ciò significa che la scelta del modello non può essere separata dall'obiettivo operativo: massimizzare la capacità di trovare difetti è diverso dal minimizzare falsi positivi o massimizzare la resa entro un budget di ispezione.

La Milestone 3 quantifica l'associazione appresa dal modello tra code smell e bugginess: azzerando solo NSmells nelle 8.933 osservazioni che contengono smell, il numero predetto di BUGGY passa da 1.723 a 1.300, con una riduzione netta di 423 osservazioni. Il valore corrisponde al 21,04% di tutte le osservazioni BUGGY del Dataset A e al 24,55% di quelle buggy in B+. Questo risultato non dimostra che eliminare gli smell elimini causalmente i bug, ma suggerisce che gli smell costituiscono un segnale sufficientemente rilevante da modificare in modo sostanziale la previsione del modello a parità delle altre metriche.

La Milestone 4 mostra il limite decisivo di una lettura puramente statica del refactoring automatico. Su PCEnhancer l'LLM riduce gli smell tra il 12,11% e il 27,37%, ma tutte le varianti falliscono la compilazione: un miglioramento Sonar non rappresenta quindi un refactoring accettabile se compromette la compilabilità o la compatibilità con le API reali del progetto. Su ListIteratorWrapper, invece, tutte le varianti compilano e superano 19/19 test; poiché C0 parte già da zero smell, l'LLM può solo preservare o peggiorare tale dimensione. C2 preserva sia LOC sia NSmells ed emerge come miglior profilo osservato, ma una singola generazione per condizione non consente di attribuire causalmente il risultato alla sola disponibilità di T_BB.

Per i practitioner emergono tre indicazioni. Primo, le metriche di qualità e la defect prediction possono aiutare a prioritizzare la manutenzione, ma devono essere interpretate come supporto decisionale e non come prova causale. Secondo, il refactoring generato da LLM richiede gate automatici obbligatori: compilazione, test di regressione e analisi statica devono essere verificati insieme. Terzo, più contesto non equivale automaticamente a un output migliore: la qualità dipende anche dalla complessità del codice, dalla compatibilità con le dipendenze e dalla variabilità del generatore.

Per la ricerca, il caso OpenJPA suggerisce un percorso di replica che integri storia del software, smell, previsione e trasformazioni generative. Le estensioni più importanti sono una validazione

temporale dei modelli, più progetti e periodi storici, un numero maggiore di classi M4 e ripetizioni multiple delle generazioni LLM. Solo con tali repliche sarà possibile separare l'effetto del contesto di test dalla variabilità intrinseca del modello e verificare quanto i risultati osservati su OpenJPA siano generalizzabili.

Nel complesso, il progetto mostra che la qualità del software non può essere sintetizzata da una singola metrica. Dataset e modelli quantitativi rendono misurabili associazioni utili, ma l'ultima decisione richiede sempre vincoli eseguibili e controlli indipendenti: un codice con meno smell può non compilare, un modello con ottima AUC può ottimizzare diversamente Precision e Recall, e uno scenario zero-smell può modificare una previsione senza dimostrare causalità. La combinazione di evidenza storica, validazione predittiva e verifica concreta delle trasformazioni è quindi il principale risultato metodologico del lavoro.

Appendice A - Tracciabilità degli artefatti principali

Questa appendice non introduce nuovi risultati; riporta i percorsi principali utilizzati per rendere il report verificabile sul repository.

Contenuto	Percorso nel repository
Setup e baseline	isw2/docs/setup.md
Milestone 1 - metodologia completa	isw2/docs/milestone1.md
Dataset A	isw2/datasets/openjpa_dataset_a.csv
Validazione Dataset A	isw2/results/dataset/dataset_a_validation.txt
Milestone 2 - metodologia e risultati	isw2/docs/milestone2.md
Output M2	isw2/results/m2/
Milestone 3 - metodologia e what-if	isw2/docs/milestone3.md
PCEnhancer M4	isw2/docs/m4/pcenhancer-m4-analysis.md
ListIteratorWrapper M4	isw2/docs/m4/listiteratorwrapper-m4-analysis.md
README di progetto	isw2/README.md
Classi considerate	isw2/classes.txt

Tabella A.1 - Artefatti principali usati nella redazione del report.

Repository pubblico: <https://github.com/Ali-Shalby/openjpa/tree/isw2-project>

Baseline release: Apache OpenJPA 4.1.1 - commit 9d2f5f8c35691c3542606bea1d23b916bcbda8d

Revisione di riferimento del report: 01e26ec9031fdc968bc873627b03e8c991e6446b